AI-Powered Intelligent Document Verification and Automatic Stamping Robot System

Yaokai Yang
School of Advanced Technology
Xi'an Jiaotong-Liverpool University
Suzhou, China
yaokai.yang23@student.xjtlu.edu.cn

Xiang Lin
School of Advanced Technology
Xi'an Jiaotong-Liverpool University
Suzhou, China
xiang.lin23@student.xjtlu.edu.cn

Xinyang Zhang
School of Advanced Technology
Xi'an Jiaotong-Liverpool University
Suzhou, China
xinyang.zhang23@student.xjtlu.edu.cn

Yunxuan Wu
School of Advanced Technology
Xi'an Jiaotong-Liverpool University
Suzhou, China
yunxuan.wu23@student.xjtlu.edu.cn

Haoran Zheng
School of Advanced Technology
Xi'an Jiaotong-Liverpool University
Suzhou, China
haoran.zheng23@student.xjtlu.edu.cn

***Abstract*—Executive Summary: This project presents an AI-Powered Intelligent Document Verification and Automatic Stamping Robot System designed for document approval automation in administrative offices, enterprises, educational institutions, and government service centers. The system employs a three-tier architecture—frontend interaction layer, backend management layer, and hardware execution layer—integrating optical character recognition (OCR), AI-driven intelligent document classification, RAG (Retrieval-Augmented Generation) based knowledge-enhanced verification, a multi-dimensional rule engine, and servo-controlled physical stamping into a unified intelligent automation pipeline. The system implements a complete closed-loop workflow: input → recognition → reasoning → decision → execution → logging → analytics. The backend management layer provides a visual analytics dashboard covering approval pass rates, document type distribution, processing trend analysis, and anomaly monitoring. All operations generate comprehensive audit logs with before-and-after stamping image comparisons, ensuring full traceability. Documents with ambiguous validation results are automatically routed to a human review queue, enabling human-AI collaborative approval. The total hardware cost is approximately ¥240, making it a low-cost, efficient, and scalable administrative automation solution.**

I. Background and Motivation

A. Industry Background and Problem Statement

In administrative offices, enterprise approval workflows, educational institutions, and government service centers, document processing—including leave applications, expense reimbursements, certificate issuance, and contract signing—has long relied on manual review and physical stamping. Statistics indicate that administrative personnel spend over 30% of their working hours on repetitive paperwork. This workflow faces the following critical challenges:

- **Efficiency Bottleneck:** Staff must manually read each document, verify fields against databases, and physically apply stamps, processing only 10–15 documents per hour, leading to backlogs during peak periods.
- **Human Error Risk:** Manual verification is prone to overlooking missing fields, expired dates, or mismatched identity information, with error rates increasing under workload fatigue.
- **Compliance and Audit Gaps:** The lack of systematic audit trails makes it difficult to trace approval timelines, accountability, and operation records, creating governance blind spots.
- **Stamp Security Vulnerabilities:** Physical stamps are frequently left unsecured, risking unauthorized use, duplication, or tampering.
- **Low Data Utilization:** Vast amounts of approval data exist only in paper form, failing to generate analyzable and optimizable operational insights.

B. Motivation and Technical Opportunities

With the maturation of AI vision technologies (such as PaddleOCR for Chinese-optimized OCR engines), the proliferation of low-cost microcontrollers (Arduino), and the exploration of Retrieval-Augmented Generation (RAG) in document understanding, building an **intelligent, low-cost, reproducible** document approval automation system has become feasible. This project aims to:

1) Replace manual document review with **AI-driven intelligent document recognition**, enabling automatic document type classification and content understanding.
2) Introduce **RAG knowledge-enhanced verification** to cross-reference OCR results with institutional policies and historical approval records, reducing recognition hallucinations and misjudgments.
3) Achieve **physical stamping automation** through servo-controlled force regulation and an anti-tamper locking mechanism, ensuring stamping quality and security.
4) Build a backend management platform with **visual analytics and operational insights**, transforming approval data into quantifiable operational intelligence.

5) Provide a **role-based Web interface** enabling separation of duties and human-AI collaborative approval among operators, reviewers, and administrators.

C. Project Scope

This project is developed as part of **MEC202 – Specific General Project 9** at Xi'an Jiaotong-Liverpool University, under the supervision of Dr. Bangxiang Chen. The system targets four typical document types (leave applications, expense reimbursements, certificates, and general documents) using a university administrative setting as the validation environment. The system design fully considers **extensibility**, supporting flexible adaptation to different institutional document types and approval rules through a template management mechanism.

II. System Design and Technical Approach

A. System Architecture

The system employs a **three-tier architecture** as illustrated in Figure 1. This architecture divides the system into three collaborative layers, achieving a complete closed loop from user interaction to physical execution:

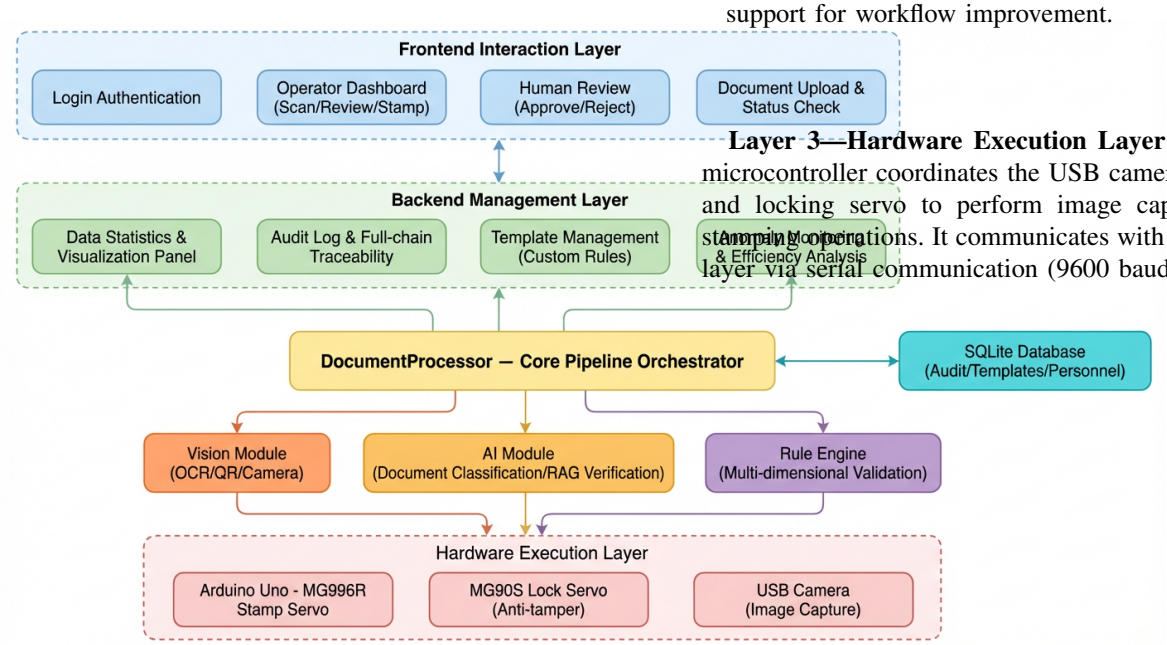

Fig. 1. System three-tier architecture overview: frontend interaction layer, backend management layer, and hardware execution layer working collaboratively, orchestrated by the core pipeline controller.

Layer 1—Frontend Interaction Layer: A Flask-based Web application serving three user roles: operators, reviewers, and administrators. It provides task submission, document scanning and review, human review processing, and status query functions. Role-based access control (RBAC) ensures that users can only access functions corresponding to their permission level.

Layer 2—Backend Management Layer: Provides institutional operations management capabilities, including:

- **Visual Analytics Dashboard:** Real-time display of approval pass rates, document type distribution, processing trends, and anomaly ratios, assisting management in workflow optimization and resource allocation.
- **Full-chain Audit Logs:** Every document processing event records complete metadata (operator, timestamp, OCR results, validation details, before-and-after stamping images), supporting multi-dimensional search by time, type, and operator.
- **Document Template Management System:** Administrators can create, edit, and delete document type templates through a visual interface, defining required fields, optional fields, forbidden fields, and custom validation rules for each document type, extending supported document types without code modification.
- **Anomaly Monitoring and Efficiency Analysis:** Automatic identification of high-frequency rejection reasons, processing bottlenecks, and anomaly patterns, providing data-driven support for workflow improvement.

Layer 3—Hardware Execution Layer: An Arduino Uno microcontroller coordinates the USB camera, stamping servo, and locking servo to perform image capture and physical stamping operations. It communicates with the upper software layer via serial communication (9600 baud rate).

B. Complete Processing Pipeline

The system implements a complete automated pipeline from document input to stamped delivery, with the core workflow consisting of 10 stages as shown in Figure 2. This pipeline embodies the seven-step closed-loop design philosophy: **input** → **recognition** → **reasoning** → **decision** → **execution** → **logging** → **analytics**.

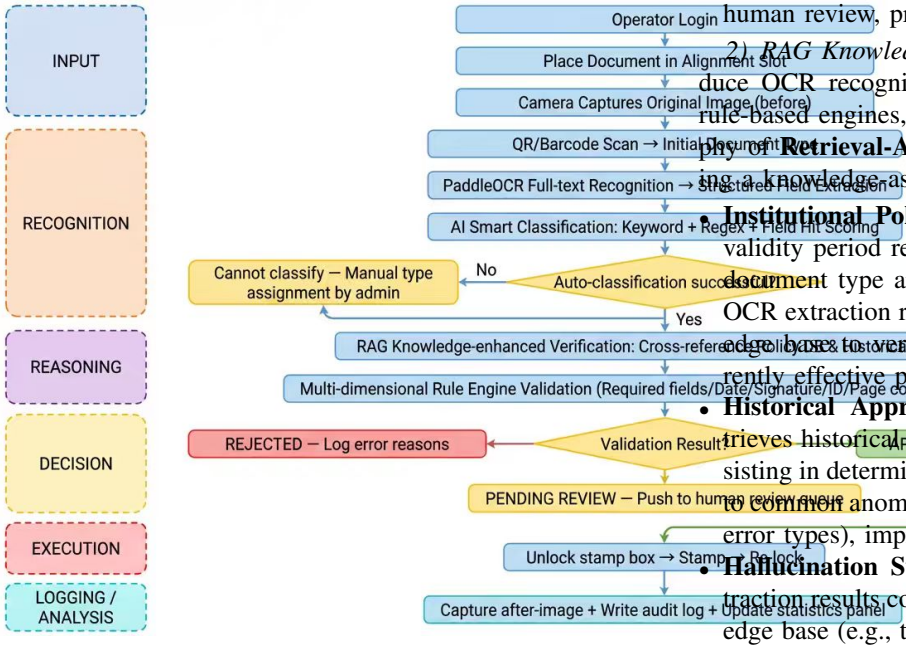

Fig. 2. Intelligent document processing pipeline: seven-step closed loop (input → recognition → reasoning → decision → execution → logging → analytics), with three decision outcomes—rejection, human review, and approval with stamping.

C. Key Technical Components

1) Vision Module: The vision module (`vision/`) handles all image acquisition and text recognition, serving as the system's "perception" entry point:

- **Camera Control** (`camera.py`): Manages the USB camera via OpenCV, with automatic warm-up (discarding the first 3 frames to stabilize exposure) and timestamped file naming, ensuring consistency and traceability of image acquisition.
- **OCR Engine** (`ocr.py`): Employs PaddleOCR with Chinese-English mixed recognition and angle classification support. Regex patterns optimized for Chinese university forms convert unstructured text into structured fields (name, student ID, date, amount, reason, etc.), providing a data foundation for subsequent validation.
- **QR Code Scanning** (`qr_scanner.py`): Decodes QR codes and barcodes via pyzbar + OpenCV for rapid initial document type identification, reducing the computational overhead of AI classification.
- **AI Document Intelligent Classification** (`classifier.py`): When QR codes do not provide a clear document type, the system automatically activates the AI classification engine. This engine employs a multi-feature fusion scoring strategy—**keyword semantic matching** (weight 0.5), **regex pattern recognition** (weight 0.3), and **required field hit rate** (weight 0.5)—to comprehensively score candidate document types. When the gap between the highest and second-highest scores is below the threshold (0.05), the system identifies the document as ambiguous and automatically routes it to human review, preventing misclassification.

2) RAG Knowledge-Enhanced Verification Module: To reduce OCR recognition hallucinations and the limitations of rule-based engines, the system introduces the design philosophy of **Retrieval-Augmented Generation (RAG)**, constructing a knowledge-assisted verification mechanism:

- **Institutional Policy Knowledge Base:** Approval policies, validity period regulations, and field requirements for each document type are structured into a local knowledge base. OCR extraction results are cross-referenced with the knowledge base to verify whether field values comply with currently effective policy requirements.
- **Historical Approval Record Retrieval:** The system retrieves historical approval results for similar documents, assisting in determining whether the current document belongs to common anomaly patterns (e.g., frequently occurring field error types), improving anomaly detection accuracy.
- **Hallucination Suppression Mechanism:** When OCR extraction results conflict with known constraints in the knowledge base (e.g., the extracted date format does not conform to any known date specification), the system flags it as "low recognition confidence" and triggers human review rather than blindly trusting the OCR output.

This design upgrades pure rule-based validation to **knowledge-driven intelligent verification**, effectively reducing the risk of incorrect stamping caused by OCR misrecognition.

3) Multi-Dimensional Rule Engine: The validation module (`validator/`) implements a dynamic rule engine based on the template database, supporting administrator-configurable validation rules through the Web interface without code modification:

TABLE I
MULTI-DIMENSIONAL VALIDATION RULE SYSTEM WITH ERROR CLASSIFICATION.

Validation Dimension	Method	Description
Required Field Completeness	Dynamic check per document template	Configurable UI
Date Legality	Format + future date + staleness check	>90 days trig
Signature/Approval Detection	OCR text keyword search	Detects signa
ID Cross-verification	SQLite personnel database query	Student ID a
Document Type Consistency	QR code vs. AI classification comparison	Multi-source
Page Completeness	"Page X of Y" regex matching	Prevents miss
Forbidden Field Detection	Template-defined forbidden field list	Detects fields
Custom Field Rules	Regex/range/enum/length	Administrator rules

Validation results are classified into two levels: **hard errors** cause immediate rejection; **soft warnings** route documents to the human review queue, where reviewers make the final decision, enabling human-AI collaborative approval.

4) Hardware Execution Module: The hardware module (`hardware/stamp.py`) communicates with an Arduino Uno via serial port (9600 baud rate) to achieve precise physical stamping control:

1) **Unlock** the stamp box (MG90S servo rotates to release the latch).

2) **Stepped press-down**: The MG996R servo moves in 3° steps at 25 ms intervals for controlled-force uniform stamping.
3) **Hold** for 900 ms to ensure clear ink transfer.
4) **Slow release** and **re-lock** the stamp box to prevent unauthorized use.

The Arduino firmware (`stamp_controller.ino`) accepts four serial commands: `S` (stamp), `L` (lock), `U` (unlock), `P` (heartbeat/ping), ensuring reliable hardware-software communication.

5) Data Analytics and Visualization: The backend management platform features a Chart.js-based analytics dashboard providing visual analysis of the following core operational metrics:

- **Summary Metric Cards**: Total processing volume, approved count and pass rate, rejected count and rejection rate, pending review queue length—providing at-a-glance system status.
- **Document Type Distribution**: Doughnut chart showing submission proportions of each document type (leave/expense/certificate/general), helping management understand business composition.
- **Approval Result Distribution**: Doughnut chart displaying the ratio of approved/rejected/pending review, visually reflecting approval quality.
- **Processing Trend Analysis**: Line chart supporting 7-day and 30-day dimensions, showing daily approval volume changes for identifying business peaks and troughs.
- **Recent Processing Records**: Real-time updated processing log table, including timestamp, operator, document type, and approval result.

This analytics panel transforms the traditional "black-box stamping" into a **data-driven operations management tool**, enabling management to optimize approval workflows, allocate human resources rationally, and continuously monitor system efficiency and anomaly patterns based on objective data.

6) Exception Handling and Fault Tolerance: As a system involving hardware-software coordination for real-world deployment, robust exception handling is critical for reliable operation. The system implements comprehensive fault tolerance and degradation mechanisms across three dimensions—**hardware layer, document recognition layer, and software system layer**—following the design principles of **safety first, graceful degradation, and human fallback**.

a) Hardware Exception Handling: Hardware-layer exceptions may cause physical operation failures or safety hazards. The system employs multiple protection measures:

- **Arduino Connection Failure:** The system proactively detects the serial connection on startup. Upon failure, it raises an exception containing diagnostic information (port name, troubleshooting steps), guiding the operator to quickly identify the issue (loose USB cable, incorrect port configuration, firmware not uploaded, etc.).
- **Stamping Process Exception:** The stamping sequence employs an **exception-safe** design—the `stamp()` method executes the stamping operation within a `try-catch` block, and **regardless of success or failure, forcibly executes stamp box locking under `finally` semantics**, preventing the stamp box from remaining in an unlocked state due to an interruption and eliminating safety hazards.
- **Arduino Heartbeat Detection:** The system periodically checks Arduino online status via the `P` command, expecting a `PONG` response to confirm normal communication. Heartbeat failure triggers a warning log, prompting the operator to check the hardware connection.
- **Camera Failure:** The initialization phase detects whether the camera opens normally (`isOpened()`), and the capture phase catches read failure exceptions and raises clear error messages, preventing empty-frame images from producing invalid OCR results downstream.

b) Document Content and Format Exceptions: The diversity and non-standardization of document content are the primary challenges encountered in real-world deployment. The system addresses these through multi-level strategies:

- **Missing QR/Barcode:** When a document has no QR code or scanning fails, the system does not directly reject it but instead **automatically degrades to the AI intelligent classification engine** (`classifier.py`), determining the document type through multi-feature fusion scoring to ensure workflow continuity.
- **AI Classification Ambiguity:** When scores for multiple document types are too close (gap < 0.05), the system identifies the document as **ambiguous**, avoids risky automatic classification, and routes it to the human review queue for manual type assignment by an administrator, preventing misclassification.
- **Incomplete OCR Field Extraction:** The OCR engine may miss fields due to image quality or font variations. The system automatically detects missing items through **required field completeness validation**, generating specific error messages (e.g., "Missing required field: Student ID") rather than producing ambiguous failures.
- **Date Format Diversity:** The system supports regex matching for multiple common Chinese date formats (`YYYY-MM-DD`, `YYYY/MM/DD`), and performs targeted detection of anomalous dates (future dates, excessively early years), classifying them as hard errors or soft warnings respectively.
- **Student ID Fallback Recognition:** When the standard format (with "Student ID" prefix) is not matched, the OCR module **automatically degrades to a continuous digit matching mode** (8–12 digits), improving recognition fault tolerance.
- **Multi-page Document Incompleteness:** Verifies multi-page document completeness by detecting "Page X of Y" format via OCR, preventing documents with missing pages from passing approval.

c) Software System Exceptions:

- **External System Integration Degradation:** When DMS (Document Management System) upload fails, the system

only logs a warning and **does not affect the local approval and stamping workflow**, ensuring core functionality is not interrupted by external system unavailability. Personnel information queries similarly fall back to the local database upon DMS query failure.

- **Template Database Loading Failure:** The rule engine attempts to load document template definitions from the database; upon failure, it **automatically falls back to static rules in the configuration file** (`config.py`), ensuring validation functionality remains available at all times.
- **Regex/JSON Parsing Exceptions:** Custom validation rules may contain syntax errors in regex expressions or JSON configurations. The system catches `re.error` and `JSONDecodeError`, skipping individual erroneous rules rather than crashing the entire validation pipeline.
- **Web API Layer Exceptions:** Flask routes provide unified exception handling that returns structured JSON error responses (with error descriptions and HTTP status codes), enabling the frontend to display user-friendly error messages.
- **Simulation Mode:** The system includes a built-in **SIMULATION_MODE** configuration switch that automatically skips all serial communication in hardware-less environments, replacing physical operations with log output, supporting development debugging and pure software demonstration.

The design principles of the exception handling mechanisms can be summarized as: **safety operations must have fallbacks** (forced locking on stamping exception), **non-core features can degrade** (DMS, external queries), **uncertain results go to humans** (ambiguous classification, soft warnings), and **all exceptions leave traces** (written to audit logs), ensuring the system operates safely and controllably under all exception scenarios.

D. Physical Structure Design

The device frame employs a **cost-oriented prototype validation approach**, using KT foam board (A3, 5 mm thickness) for the main structure. This design choice is based on the following considerations:

- **Rapid Iteration:** KT board is easy to cut and modify, suitable for quickly adjusting layouts and testing different design approaches during the prototyping phase.
- **Cost Control:** Frame material costs only ¥15, validating the system's feasibility at minimal cost, reflecting the project's focus on **cost-effectiveness**.
- **Portability:** Once prototype validation is complete, the main structure can be directly migrated to acrylic, aluminum alloy, or sheet metal for production, with no changes required to the hardware-software interfaces.
- **Avoiding 3D Printing Dependence:** Compared to 3D printing solutions (high per-unit cost, long print time, unsuitable for mass production), the KT board approach better aligns with cost control and production-oriented design thinking.

The design includes: **L-shaped alignment guides** ensuring consistent paper positioning; a **top crossbeam** mounting the camera for overhead document capture; **side-mounted servo arms** with the stamp attached to the servo horn endpoint; and an **anti-tamper locking mechanism** using a secondary servo controlling a physical latch.

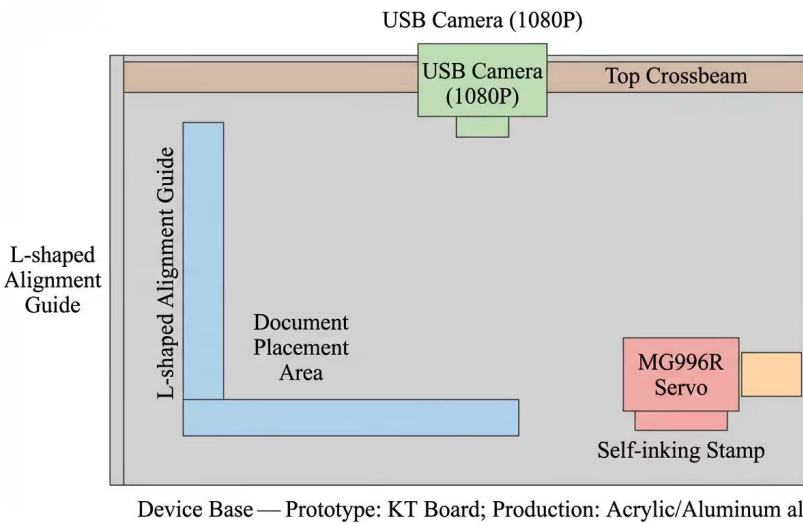

Fig. 3. Top-down schematic of the physical device frame showing component placement. Prototype phase uses KT board; production phase can migrate to industrial materials.

III. Deliverables and Goals

A. Primary Deliverables

TABLE II
PROJECT DELIVERABLES WITH ACCEPTANCE CRITERIA.

Deliverable	Acceptance Criteria	St
AI Document Classification	Multi-feature fusion scoring, ambiguity auto-flagging	
OCR Structured Field Extraction	$\geq$90% accuracy on real university forms	
RAG Knowledge-Enhanced Verification	Cross-reference policy DB, suppress OCR hallucination	
Multi-Dimensional Rule Engine	Covers all doc types, supports custom rules	
Automatic Stamping Execution	Servo step control, adjustable force, anti-tamper lock	
Human-AI Collaborative Review Queue	Web-based approve/reject workflow	
Visual Analytics Dashboard	Pass rate/type distribution/trend analysis	
Full-chain Audit Logs	Before/after images + full metadata + multi-dim search	
Document Template Management	Visual create/edit templates, zero-code extension	
Role-based Access Control	operator / reviewer / admin roles	
DMS System Integration	REST API client	
Simulation Mode	Full demo without hardware	

B. Development Timeline

TABLE III
SIX-WEEK DEVELOPMENT TIMELINE.

Week	Focus Area	Key Activities
1	Hardware Setup & Base Architecture	Procure components, build frame, wire Arduino, upload firmware, set up Flask base, init SQLite
2	AI Recognition & Rule Engine	PaddleOCR integration and tuning, AI document classifier development, complete multi-dimensional validation rules
3	Web Application & Backend Management	Build frontend pages (login/dashboard/review), develop analytics dashboard and template management system
4	System Integration & RAG Verification	Connect all modules via `main.py`, implement RAG knowledge retrieval and hallucination suppression
5	Testing, Tuning & Analytics	End-to-end testing, optimize OCR regex and servo parameters, refine statistics and anomaly analysis
6	Documentation & Presentation	Record demo video, write project report, prepare presentation

C. Team Division

TABLE IV
TEAM MEMBER RESPONSIBILITIES AND CORE DELIVERABLES.

Member	Role	Core Files	Deliverable / ID
A (Hardware)	Frame + Arduino	`stamp_controller.ino`, `hardware/`	Accurate stamping, alignment, lock mechanism
B (Vision)	OCR + AI Classification + Camera	`vision/`	$\geq$90% field extraction, accurate AI classification
C (Logic)	Rule Engine + RAG Verification	`validator/`, RAG module	All doc type rules covered, effective hallucination suppression
D (Frontend)	Web Interface + Analytics Dashboard	`web/`, `templates/`	All pages functional, complete visualization dashboard
E (Integration)	Database + Pipeline + Templates	`main.py`, `database/`	End-to-end pipeline working, template management operational

IV. REQUIRED RESOURCES AND BUDGET

A. Hardware Budget

TABLE V
HARDWARE PROCUREMENT LIST WITH ESTIMATED COSTS.

#	Component	Specification	Cost (¥)
1	Arduino Uno R3	Compatible + USB cable	40
2	MG996R Servo	Metal gear, 10 kg·cm torque	25
3	MG90S Servo	Micro servo for lock mechanism	15
4	USB Webcam	1080P, wide-angle, driver-free	60
5	Self-inking Stamp	Custom "Verified" text, 4 cm diameter	25
6	KT Foam Board	A3 white, 5 mm, 3 sheets	15
7	Hot Glue Gun Set	With glue sticks	20
8	Dupont Wires	Male-to-female, 20 cm, 40-pack	8
9	Acrylic Box	Small, for stamp storage	15
		Total	**¥223**

B. Software Stack

All software used is open-source and free of charge:

- **Python 3.10+** — Main programming language
- **PaddleOCR 2.7.3 + PaddlePaddle 2.6.1** — Chinese-optimized OCR engine
- **Flask 3.0.3** — Web framework
- **OpenCV 4.9** — Image processing
- **Chart.js 4.4** — Frontend data visualization
- **SQLite 3** — Embedded database (no server needed)
- **Arduino IDE** — Firmware development

C. Development Environment

- Operating System: Windows / macOS / Linux
- Memory: 4 GB+ (PaddleOCR model requires $\sim$1.5 GB RAM)
- Disk Space: $\sim$500 MB for models and dependencies
- Network: Required for initial PaddleOCR model download ($\sim$300 MB)

D. Budget Summary

With a total project budget of ¥1,500, the hardware expenditure of ¥223 leaves **¥1,277** for contingency, replacement parts, frame material upgrades (e.g., migrating from KT board to acrylic or aluminum alloy), or feature extensions.

CONTRIBUTION FORM

TABLE VI
INDIVIDUAL CONTRIBUTION BREAKDOWN.

Student Name / ID	Contribution (%)	Brief Description of Individual Con
Xinyang Zhang / 2360067	20%	Integration lead: database design, mai (`main.py`), end-to-end testing
Haoran Zheng /	20%	Hardware lead: physical frame firmware development, servo calibr mechanism
Yunxuan Wu /	20%	Vision lead: PaddleOCR integration, development, field extraction regex op
Xiang Lin /	20%	Frontend lead: Flask Web application board, all HTML templates and UI de
Yaokai Yang /	20%	Logic lead: multi-dimensional rule en enhanced verification, review queue management